

CAHIER DES CHARGES v2.0

CONFIDENTIEL — Document de référence projet

Chambres Meublées à Douala

avec Assistant Conversationnel IA

Version

2.0 — Optimisé

Statut	En cours de développement
Cible	Douala, Cameroun
Stack	Next.js · NestJS · PostgreSQL · OpenAI API

Table des matières

1.	Contexte et problématique
2.	Étude de l'existant
3.	Objectifs du projet
4.	Acteurs et rôles
5.	Fonctionnalités détaillées
6.	Architecture technique recommandée
7.	Modèle de données
8.	Sécurité et performances
9.	Contraintes et exigences
10.	Évolutions futures
11.	Méthodologie et livrables

Contexte et Problématique

La ville de Douala, capitale économique du Cameroun, concentre une forte demande en logements meublés de courte et moyenne durée. Malgré cette demande croissante, le marché reste peu structuré et quasi intégralement informel.

Canaux actuels de recherche de logements :

- Bouche-à-oreille et réseaux personnels
- Groupes WhatsApp sans modération ni garantie
- Facebook Marketplace — absence de système de réservation
- Agences immobilières traditionnelles — lentes, coûteuses

Problèmes identifiés

- Aucune centralisation des offres — le chercheur doit consulter de multiples sources
- Informations non fiables, photos inexistantes ou trompeuses
- Impossibilité de réserver en ligne — tout se fait par appel ou déplacement physique
- Pas d'assistance automatisée — disponibilité des propriétaires limitée
- Faible visibilité des logements de qualité face aux arnaques

Problématique centrale :

Comment concevoir une plateforme web moderne permettant de centraliser les offres de chambres meublées à Douala, de faciliter les réservations en ligne, d'automatiser l'assistance client grâce à l'intelligence artificielle, et de garantir une expérience utilisateur fluide pour plusieurs utilisateurs simultanés ?

Étude de l'Existant

Plateformes internationales de référence :

Plateforme	Points forts	Limites au contexte local
Airbnb	UX excellente, paiements sécurisés, avis vérifiés	Très peu de logements camerounais. Paiement par carte uniquement. Interface en anglais dominant.
Booking.com	Large catalogue, filtres avancés	Idem Airbnb. Commission élevée pour les propriétaires. Pas adapté aux locations informelles.
Facebook Marketplace	Très utilisé localement, gratuit	Aucune garantie. Pas de réservation. Arnaque fréquente. Zéro automatisation.

Opportunité identifiée

Le marché local est structurellement sous-équipé en outils numériques. Aucune plateforme spécialisée n'existe pour Douala avec : paiement Mobile Money, interface en français, assistant IA en langue locale, et adaptation aux réalités camerounaises (connexion variable, usage mobile dominant). L'opportunité est directe et le besoin réel.

Objectifs du Projet

Développer une plateforme web de réservation de chambres meublées à Douala, multi-utilisateurs, avec un assistant conversationnel IA intégré nativement au backend.

Objectifs MVP (v1.0)	Objectifs v2.0 (6-12 mois)
• Consultation et recherche de logements • Système de réservation avec confirmation • Chatbot IA textuel (recherche + recommandation) • Espace propriétaire avec CRUD logements • Interface administrateur complète • Authentification sécurisée multi-rôles • Responsive mobile-first	• Paiement Mobile Money (MTN MoMo, Orange Money) • Notifications WhatsApp automatiques (n8n) • Système d'avis et notation • Assistant vocal (OpenAI Whisper + GPT-4o Audio) • Application mobile (React Native) • Géolocalisation avancée • Multi-agences / marketplace

Acteurs et Rôles

La plateforme distingue quatre rôles avec des droits distincts. Cette séparation est fondamentale pour la sécurité et la scalabilité.

Visiteur (non connecté)
• Consulter la liste des logements disponibles
• Voir les détails d'un logement (photos, prix, équipements)
• Utiliser le chatbot IA (recherche uniquement)
• S'inscrire ou se connecter
Locataire (utilisateur connecté)
• Toutes les fonctions du visiteur
• Effectuer une demande de réservation
• Suivre ses réservations (historique, statut)

• Communiquer avec le propriétaire via la plateforme
• Laisser un avis (v2.0)
Propriétaire / Bailleur (rôle ajouté v2.0 optimisé)
• Ajouter, modifier, supprimer ses propres logements (CRUD)
• Gérer les disponibilités de ses logements
• Consulter et répondre aux demandes de réservation
• Télécharger les photos via Cloudinary
• Voir les statistiques de ses logements
• Ne peut pas voir les logements des autres propriétaires
Administrateur (super-admin)
• Toutes les fonctions des autres rôles
• Valider les comptes propriétaires
• Modérer tous les logements et toutes les réservations
• Accéder aux statistiques globales de la plateforme
• Gérer les utilisateurs (activation, suspension)
• Configurer les paramètres globaux du système

Fonctionnalités Détaillées

5.1 Consultation et recherche de logements

La page principale affiche les logements disponibles sous forme de cartes avec photo principale, quartier, prix/nuit et disponibilité. Le moteur de recherche permet de filtrer par :

- Quartier (Akwa, Bonanjo, Deido, Bonapriso, etc.)
- Fourchette de prix (en FCFA)
- Dates de disponibilité
- Type de logement (chambre, studio, appartement)
- Équipements (climatisation, WiFi, parking, cuisine)

5.2 Système de réservation

Étape	Action utilisateur	Action système
1	Choisit un logement	Vérifie disponibilité en temps réel
2	Sélectionne les dates	Calcule le montant total
3	Soumet la demande	Notifie le propriétaire
4	Attend confirmation	Propriétaire accepte ou refuse
5	Reçoit notification	Statut mis à jour, logement bloqué

5.3 Assistant IA conversationnel

L'agent IA est intégré nativement dans le backend NestJS. Il utilise l'API OpenAI avec fonction calling pour interroger directement PostgreSQL sans passer par un outil tiers. Chaque utilisateur bénéficie d'une session indépendante, ce qui garantit la performance en contexte multi-utilisateurs.

Fonctionnement technique de l'agent IA

1. L'utilisateur envoie un message : « Je cherche une chambre a Akwa a moins de 25 000 FCFA »
2. Le backend NestJS transmet le message a l'API OpenAI GPT-4o avec fonction calling
3. L'IA appelle la fonction `search_rooms({ quartier: 'Akwa', prix_max: 25000 })`
4. NestJS execute la requete SQL sur PostgreSQL et retourne les resultats
5. L'IA formule une reponse naturelle en francais avec les logements correspondants
6. L'utilisateur voit les logements directement dans l'interface du chatbot

Capacités de l'agent IA — MVP :

- Comprendre les demandes en français (et anglais basique)
- Rechercher des logements selon les critères exprimés naturellement
- Recommander des alternatives si aucun logement ne correspond exactement
- Répondre aux questions sur les équipements, le quartier, les prix
- Guider l'utilisateur vers le formulaire de réservation
- Maintenir le contexte de conversation (mémoire de session)

Pourquoi pas n8n/Make comme cerveau agent ?

Ces outils d'automatisation ne sont pas conçus pour gérer des sessions simultanées de multiple utilisateurs. Chaque requête crée un goulot d'étranglement, la latence s'accumule, et le débogage devient très difficile. Ils sont réservés aux automatisations périphériques (notifications, emails). L'agent IA central vit dans NestJS.

Architecture Technique Recommandée

6.1 Vue d'ensemble

Architecture trois tiers classique, avec l'agent IA intégré comme service interne du backend. Aucun orchestrateur externe (n8n, Make, LangChain) n'est utilisé pour le flux principal — uniquement l'API OpenAI en direct.

Couche	Technologie	Rôle
Frontend	Next.js 14 + React + Tailwind CSS	Interface utilisateur, SSR/SSG, Vercel AI SDK pour le streaming du chatbot
Backend	NestJS (Node.js) + TypeScript	API REST, logique metier, agent IA integre, gestion des roles et sessions
Base de données	PostgreSQL + Prisma ORM	Stockage persistant, requetes optimisees, migrations versionnees
Stockage médias	Cloudinary	Upload, compression et CDN des photos de logements
Intelligence artificielle	OpenAI GPT-4o (API directe)	Chatbot textuel avec fonction calling, futur : Whisper pour la voix
Authentification	JWT + bcrypt + OTP SMS	Tokens securises, hash des mots de passe, verification par telephone
Automations périph.	n8n (auto-hébergé)	Notifications WhatsApp, emails transactionnels — usage peripherique uniquement

6.2 Stack technologique détaillée

Next.js 14 v14.x Frontend React SSR	NestJS v10.x Backend TypeScript	PostgreSQL v15+ Base de données
Prisma v5.x ORM & migrations	OpenAI GPT-4o API directe Agent IA natif	Cloudinary SDK v2 Médias & CDN

6.3 Flux de données — Agent IA multi-utilisateurs

Chaque utilisateur ouvre une session de conversation indépendante. Le backend NestJS traite les requêtes en parallèle via le gestionnaire d'événements asynchrones de Node.js. L'historique de conversation est stocké en base de données et reconstruit à chaque appel API OpenAI pour maintenir le contexte.

Pourquoi cette approche tient à la charge

- Node.js traite les I/O de manière non-bloquante — 100 utilisateurs en chat simultané ne saturent pas le CPU
- Chaque session appelle l'API OpenAI indépendamment — pas de goulot d'étranglement centralisé
- PostgreSQL avec connection pooling (PgBouncer) gère des centaines de connexions simultanées
- Le déploiement sur Railway ou Render permet le scaling vertical en un clic

Modèle de Données

Cinq entités principales constituent le cœur du modèle relationnel. Les clés étrangères garantissent l'intégrité référentielle. Prisma ORM génère automatiquement les migrations.

Entite : Utilisateur (User)

Champ	Type	Description
id	UUID	Identifiant unique, généré automatiquement
nom	VARCHAR(100)	Nom complet
email	VARCHAR(255)	Email unique, vérifié
telephone	VARCHAR(20)	Numéro camerounais — requis pour Mobile Money et OTP
mot_de_passe	VARCHAR(255)	Hashé avec bcrypt (min 12 rounds)
role	ENUM	VISITEUR LOCATAIRE PROPRIETAIRE ADMIN
photo_profil	VARCHAR(500)	URL Cloudinary
est_verifie	BOOLEAN	Compte validé par email/SMS
cree_le	TIMESTAMP	Date de création

Entite : Logement (Property)

Champ	Type	Description
id	UUID	Identifiant unique
proprietaire_id	UUID FK	Référence vers User (rôle PROPRIETAIRE)
nom	VARCHAR(200)	Titre de l'annonce
description	TEXT	Description complète
quartier	VARCHAR(100)	Quartier à Douala
adresse	TEXT	Adresse détaillée
prix_nuit	INTEGER	Prix en FCFA
type	ENUM	CHAMBRE STUDIO APPARTEMENT VILLA
equipements	JSONB	Liste des équipements (WiFi, clim, parking...)
images	JSONB	Tableau d'URLs Cloudinary
disponible	BOOLEAN	Disponibilité courante
cree_le	TIMESTAMP	Date de création

Entite : Réservation (Booking)

Champ	Type	Description
id	UUID	Identifiant unique
locataire_id	UUID FK	Référence vers User (LOCATAIRE)
logement_id	UUID FK	Référence vers Property
date_debut	DATE	Date d'arrivée souhaitée
date_fin	DATE	Date de départ souhaitée
montant_total	INTEGER	Total calculé en FCFA
statut	ENUM	EN_ATTENTE ACCEPTEE REFUSEE ANNULEE TERMINEE
message	TEXT	Message optionnel du locataire
cree_le	TIMESTAMP	Date de la demande

Entite : Conversation IA (ChatSession)

Champ	Type	Description
id	UUID	Identifiant de session
utilisateur_id	UUID FK	Référence vers User (nullable pour visiteur)
historique	JSONB	Tableau des messages {role, content, timestamp}
cree_le	TIMESTAMP	Début de session
mis_a_jour_le	TIMESTAMP	Dernier message

Sécurité et Performances

8.1 Authentification et autorisation

- JWT (JSON Web Tokens) avec expiration courte (1h) + refresh token (7 jours)
- Mots de passe hashés avec bcrypt (minimum 12 rounds de salage)
- Vérification du compte par OTP SMS sur numéro camerounais
- Guards NestJS sur chaque route selon le rôle requis (RBAC)
- Rate limiting sur les endpoints sensibles (login, inscription, chatbot)

8.2 Protection des données

- Validation stricte de toutes les entrées avec class-validator (NestJS)
- Protection contre les injections SQL via Prisma ORM (requêtes paramétrées)
- Headers de sécurité HTTP avec Helmet.js
- Variables d'environnement pour toutes les clés API (jamais en dur dans le code)
- HTTPS obligatoire en production (certificat Let's Encrypt)
- Clé API OpenAI côté serveur uniquement — jamais exposée au frontend

8.3 Performances pour usage simultané

Cibles de performance

- Temps de réponse API : < 300ms pour 95% des requêtes hors IA
- Réponse chatbot IA : < 3 secondes (streaming activé pour affichage progressif)
- Disponibilité cible : 99.5% (acceptable pour un MVP en phase de lancement)
- Utilisateurs simultanés cibles phase 1 : 50-100 sessions actives
- Images : compression automatique Cloudinary, formats WebP, lazy loading frontend

Contraintes et Exigences

Contraintes techniques

- Responsive mobile-first (80% des utilisateurs sur mobile)
- Fonctionne avec connexion 3G (images optimisées, lazy loading)
- Support Chrome, Firefox, Safari mobile minimum
- Internationalisation : français par défaut
- Accessibilité : contrastes suffisants, textes lisibles

Contraintes métier

- Le propriétaire gère uniquement ses propres logements
- Une réservation ne peut être acceptée que par le propriétaire du logement
- Un logement accepté bloque automatiquement les dates
- L'admin peut tout voir et tout modifier
- Les prix sont en FCFA (Franc CFA d'Afrique Centrale)

Évolutions Futures

Horizon	Fonctionnalité	Technologie envisagée
6 mois	Paieement Mobile Money	MTN MoMo API, Orange Money API
6 mois	Notifications WhatsApp	n8n + WhatsApp Business API
6 mois	Système d'avis et notation	Entité Review en base, frontend 5 étoiles
12 mois	Assistant vocal	OpenAI Whisper (STT) + GPT-4o Audio (pas Vapi)
12 mois	Application mobile	React Native (code partagé avec Next.js)
12 mois	Géolocalisation	Google Maps API ou Mapbox (open source)
18 mois	Multi-agences / marketplace	Extension du modèle propriétaire, commissions
18 mois	IA vocale temps réel	OpenAI Realtime API (GPT-4o Realtime)

Note sur Vapi et n8n/Make

Vapi est un excellent outil pour les voice bots telephoniques. Il n'est pas adapte a un chatbot web embarque avec connexion 3G variable depuis Douala : latence elevee, facturation a la minute imprevisible, complexite d'integration inutile au stade MVP. L'integrer a l'horizon 12-18 mois, uniquement si un canal telephonique est requis.

n8n reste recommande pour les automations peripheriques uniquement (notifications, emails, webhooks). Il ne doit jamais etre le cerveau central de l'agent IA.

Méthodologie et Livrables

11.1 Approche de développement

Développement agile itératif avec sprints de 2 semaines. Chaque sprint livre une fonctionnalité testable et déployable.

Sprint	Livable	Durée
1	Setup projet, BDD, authentification multi-rôles	2 semaines
2	CRUD logements (propriétaire + admin), upload Cloudinary	2 semaines

3	Interface publique, recherche et filtres	2 semaines
4	Système de réservation complet	2 semaines
5	Agent IA chatbot avec fonction calling	2 semaines
6	Tests, corrections, déploiement production	2 semaines

11.2 Livrables attendus

- Application web fonctionnelle déployée (frontend Next.js + backend NestJS)
- Base de données PostgreSQL avec schéma versionné via Prisma Migrate
- Documentation technique complète (API Swagger/OpenAPI auto-générée)
- Documentation utilisateur (guide propriétaire + guide locataire)
- Code source versionné sur GitHub avec README de déploiement
- Interface administrateur opérationnelle
- Chatbot IA intégré et fonctionnel en français
- Rapport de tests (unitaires, intégration, charge)

Ce cahier des charges constitue le document de référence du projet. Il peut évoluer en cours de développement selon les retours utilisateurs et les contraintes techniques identifiées. Toute modification majeure doit faire l'objet d'une version mise à jour.